

ClusterAttention: A training-free routing method for sparse bidirectional attention

Kasper Nordenram* Amelie Dittmann*

*Independent researcher

June 16, 2026

Abstract

This paper introduces ClusterAttention, a training-free wrapper for bidirectional attention layers. Through recursive splitting in transformed spaces informed by the queries for the keys and vice-versa, it separately groups the keys and queries of each attention-head into localized clusters of a fixed size. Each query-cluster is then assigned a set of key-clusters based on cluster-level scoring, and attention is performed between the tokens in the clusters. The clustering is done in sizes that are powers of 2, allowing for optimal tiling on a GPU. ClusterAttention operates on any set of key-value-pairs and queries, without requiring the input to have any structure, such as a meaningful ordering. It targets large token-counts and models with meaningfully sparse attention patterns, where the overhead of accurate clustering is outweighed by the savings from highly sparse attention.

Preliminary evaluation has been carried out on DINOv2 with SpargeAttn as baseline [1]. We evaluate the Pareto front of latency on a Nvidia H100 GPU vs. distortion of image embeddings compared to dense attention. We observe that while ClusterAttention does have meaningful routing overhead, the impact of this diminishes as the number of tokens grows. At 360,000 tokens, the more accurate routing of ClusterAttention makes it Pareto-better than SpargeAttn for much of the front. Since ClusterAttention targets token counts on the scale of one million and more, this is a promising indication, although more extensive testing is required before drawing conclusions.¹

¹This is a preprint, and method development has not concluded. More extensive evaluation is planned. This report will be updated as work progresses, and the code will become available on <https://github.com/SpoketKasper/ClusterAttention>.

1 Preliminaries

Bidirectional attention is a commonly occurring operation in modern transformer-based models, where no causal structure is necessary. Examples of this are vision transformers, video generation models, (many) text embedding models, genomic models, and models for tabular data. However, a major drawback of this operation (shared with causal attention) is that it has a computation-cost that is quadratic in the number of tokens, as each token "attends" to all tokens (including itself). Often, however, many of these connections are weak, and do not meaningfully influence the output of the operation.

Methods for cheaply pruning these interactions, and performing the attention operation from each token only onto a subset of tokens where the connection matters, are called *sparse-attention methods*, and the full attention computation in contrast referred to as *dense*. The process of deciding which queries attend to which keys and how is in this work referred to as *routing*. When a certain attention interaction between a key and a query happens directly, with a single token of resolution, this is referred to as performing per-token attention between these tokens in this work. The optimal routing of attention connections, such that the highest possible amount of attention-mass is retained for a given number of tokens attended to, is identified as *oracle* routing. Methods that can be applied at inference time as a simple modification to pretrained attention layers are referred to as *training-free* since they can be used with no further training, although training with the modification in place may still be possible.

In some cases, such as tabular data [2], high-resolution (for example pathology) imagery [3], video generation [4], and genome data [5], large token counts are common. Approaches to work in these domains often set limits to the token count, accept the high latency, or fundamentally alter the attention computation. An alternative path toward making these cases computationally tractable is using sparse attention, and accurately approximate the bidirectional attention instead of changing its core computation. This work targets this approach to these domains. On a high level, the method replaces attention between all keys and queries with the following steps:

1. Cluster keys and queries
2. Score clusters against each other, and select a subset of key-clusters for each query-cluster
3. Perform attention over the selected clusters

This can provide a meaningful speedup over dense attention if the routing overhead is small compared to the saved attention computation. Specifically, this requires that the clustering is relatively fast, clusters are large enough for the quadratic scoring and selection to be fast (or the selection is made to be sub-quadratic), and attention is not meaningfully slower per key-query-interaction in the sparse form.

2 Related works

SpargeAttn is a training-free method for sparsifying attention [1]. It relies on structure in the input data, such as a meaningful ordering, or space-filling curves in image or video space to create clusters. SpargeAttn has in this paper been used with row-major clustering for images, as an official implementation of space-filling-curve form was not found. The difference between row-major and space-filling-curve forms also does not seem to be large in the SpargeAttn paper. SpargeAttn selects clusters such that their sizes are powers of 2, to work well with GPU tiling. Furthermore, they measure the internal variance of clusters. If it falls above a threshold, the cluster is included in all computations, as centroid based routing may be inaccurate.

Clustered Attention is a method that clusters queries per-head, and uses the centroid of each query cluster to represent all the queries in the cluster [6]. Vyas et al. both apply the method to pretrained models, and train new models with it in place. They use a fast clustering method, that uses locality-sensitive hashing on the queries, and then performs K-Means in the Hamming space. They also improve on this approximation by, for each query cluster, computing dense attention over the k keys with the highest attention weight for each query cluster.

AdaCluster is a training-free method for that separately clusters keys and queries per-head [7]. Tan et al. note that keys and queries play different roles in attention, and based on this argue for different clustering methods for them. Keys are clustered in Euclidean space using a custom Multi-stage K-Means algorithm, while queries are normalized before normal K-Means clustering to make it angle-based, which they find gives more compact clusters. For assigning key-clusters to query-clusters they use TensorQuest, a modification of Quest [8]. The method explicitly targets video diffusion transformers, and although Tan et al. motivate some design choices by observations in video generation models, the method may be applicable to other domains using bidirectional attention as well.

SVOO is a training-free method that does offline layer-wise sparsity profiling [9]. Luo et al. note that in video diffusion transformers, sparsity is mostly independent of input, and rather an intrinsic property of each layer. They note that two keys are similar from the perspective of the queries if they yield similar attention logits over all queries, while queries are similar from the perspective of the keys if they have similar dot products with all keys. They use these observations to design a clustering method that performs iterative refinement of randomly initiated clusters of keys and queries. For assignment of key-clusters to query-clusters, they use the dot products of the cluster centroids. The keys and queries are reclustered every $N = 20$ diffusion steps. Similarly to AdaCluster, the method explicitly targets video generation, but may be more broadly applicable.

ClusterAttention shares several ideas present in these works. It uses clustering that takes into account the different roles of keys and queries as in AdaCluster, and the interactions of keys and queries as in SVOO. It also uses tiling-adapted cluster sizes as in SpargeAttn. In general, this subset of sparse attention methods follow a similar pattern of clustering keys and/or queries, selecting which key-query interactions to skip using these clusters, and then performing a sparse attention operation. Where they differ is the details on how each of these parts is carried out. ClusterAttention uses derived transformations of the

key and query spaces that take into account their interactions, together with recursive splitting along the principal components to produce clusters of the predetermined sizes. Furthermore, ClusterAttention makes no assumptions about the structure of the input.

3 Method

The method was developed through a combination of theoretical analysis and iterative experimentation on extracted attention scores from the original TabPFN [10], on synthetic datasets from the TabICL prior generator [11].

3.1 Overview

ClusterAttention consists of three parts. The first is where keys and queries are clustered (referred to as the *clustering*), the second is where which blocks in the attention matrix will be processed is decided (referred to as the *assignment*), and the third is where the attention computation is performed (referred to as the *attention*). Clustering is done for each combination of a key and a query head individually, meaning that for multi-head attention there is one clustering per attention head, and for grouped-query attention there is one clustering per query head. Assignment and attention also happen at this resolution. Complexity calculations are also per-head.

The clustering is carried out using a recursive splitting method, that projects a set of keys or queries onto an approximation of their first principal component, and partitions them by setting a threshold such that the number of tokens below the threshold is a multiple of c , the predecided cluster size. This ensures that there will be at most one cluster that is not of size c . When n is divisible by c , all final clusters are of size c , and otherwise, there is one cluster that absorbs the remainder. This is then padded with arbitrary values to size c for further operations, where the padding is masked out so it does not affect outputs.

The assignment is done by, for each cluster of queries, finding the dot product between their centroid and that of all cluster of keys. A correction taking into account the variance of the clusters may be applied. The clusters of keys are ranked according to this, and either the top- k are selected, or the attention mass in each key-clusters is estimated and the number of clusters required to pass an attention-mass recall threshold are selected.

Attention is carried out using the block-sparse attention kernel from SpargeAttn [1]. We use this to avoid the work of writing a high-performance block-sparse attention kernel. Since this kernel only allows the cluster sizes 128 for keys and 64 for queries, we use these in our evaluation.

3.2 Recursive splitting

The splitting is a variation on principal direction divisive partitioning (PDDP)[12]. *Split* here refers to partitioning vectors into two groups, based on which side they fall of a hyperplane. At each split, a set $\mathcal{S}$ of vectors is subsampled to $\min\{ad, |\mathcal{S}|\}$ where d is the dimensionality of the vectors, and a is some constant, which can be assumed to be 1 in our implementation. The first principal component is estimated using power iteration

on this subset. All vectors are then projected onto this estimate, and split such that the number of vectors below the projection threshold is the multiple of c closest to splitting the set in halves. The splitting uses a radix-based partitioning method. The splitting is repeated recursively, until all sets or all but one set contain c vectors. The power iteration is warm-started from the mean of the principal component estimate of the parent set, and a vector of random normal distributed values, with roughly the same norm as the parent estimate. The splitting is applied to keys and queries separately.

The computational complexity of this algorithm is

$$\mathcal{O}\left(pd^2\frac{n}{c} + nd\log_2(n/c)\right). \quad (1)$$

as derived in Appendix C.1. However, the part with the highest latency in the current implementation is the partitioning, which is of order $\mathcal{O}(n\log_2(n/c))$.

We now describe three variations on this splitting method, where the first two are combined for the method used in evaluation. The combination transforms to target-space before diagonalizing and finally splitting into clusters.

3.2.1 Diagonalized recursive splitting

Instead of approximating the first principal component at every split, we can compute the principal components of the full dataset, and project it onto these. This is equivalent to performing a singular-value decomposition of the data-matrix. Then, at every split, a subsample can be used to find which principal component (which now simply corresponds to a coordinate) has the highest variance, and split along that. This is faster end-to-end as it avoids power iteration and projection, but also gives a lower variance estimation problem at every split. It cannot, however, produce optimal partitionings at a given split unless the first principal component of the set of points happens to align with one of the global principal components. Empirically, it both gives better splits than extensive power iteration on a subsample of $10d$ vectors, and runs faster than brief power iteration on a subsample of d vectors.

The computational complexity is

$$\mathcal{O}(nd^2 + d^3), \quad (2)$$

which is derived in C.2.

3.2.2 Target-space clustering

We can improve the usefulness of clusters by considering their downstream usage and interactions.

Key clustering The basic recursive splitting of keys is performed along the PC1 in the Euclidean space that the keys inhabit. It is worth noting that attention keys and queries often only occupy a subspace of R^d , are not necessarily uniformly dispersed, and may have differing distributions. This means that some directions may be more meaningful to cluster densely along than others. To address this, we make the following observation.

Two keys are similar from the perspective of attention if they produce similar logits for all queries, i.e.

$$k_1 \cdot q \simeq k_2 \cdot q, \forall q \in \mathbf{Q}. \quad (3)$$

where $\mathbf{Q}$ is the set of queries for an attention head in a given forward pass. If we arrange the n queries into the query matrix Q , we see that the average squared difference between the logits of two keys over $\mathbf{Q}$ is

$$\|Qk_1 - Qk_2\|^2/n = \|Q(k_1 - k_2)\|^2/n. \quad (4)$$

We can rewrite this and define a metric, or possibly a pseudo-metric if Q does not have full column rank,

$$(k_1 - k_2)^T \frac{Q^T Q}{n} (k_1 - k_2) := d_q(k_1, k_2)^2. \quad (5)$$

from which we can define the (pseudo-)metric matrix

$$M = \frac{Q^T Q}{n}. \quad (6)$$

It follows that the inner product between two keys in this space is

$$\langle k_1, k_2 \rangle = k_1^T M k_2. \quad (7)$$

M is positive semi-definite as $x^T Q^T Q x = \|Qx\|^2 \geq 0$, so we can take the square root of it to produce R_q . Since queries often span only a subspace of $\mathbf{R}^d$, it is indeed likely that Q does not have full column rank, making M a pseudo-metric. This means that distinct keys can have a d_q of 0, which in this case is a useful property, as it inhibits the clustering from happening along dimensions that do not matter for attention. R_q is found using eigenvalue decomposition. For numerical reasons, small negative eigenvalues can arise, so these are clamped to 0. Using this, we can write the inner product as

$$\langle k_1, k_2 \rangle = k_1^T R_q^T R_q k_2 = (R_q k_1) \cdot (R_q k_2). \quad (8)$$

As is seen, we can project all keys through R_q to get to query-space, where we can run the recursive split as before using the normal dot product. Computation of R_q is done on-the-fly with actual queries rather than through pre-calibration.

Query clustering For clustering of queries, the reverse metric (i.e. with $\frac{K^T K}{n}$ instead of $\frac{Q^T Q}{n}$) is not necessarily optimal. We can consider two cases.

If selecting an adaptive number of key-clusters to approximately get a certain amount of attention-mass recall, we seek to cluster queries that would select the same key-clusters to get to a certain level of recall, i.e. that have similar attention distribution over the key-clusters. In this case, the same metric may seem a well-motivated choice, however, the softmax operation complicates this. We can show this with the following argument. For two keys attended to by a query, them having the same dot product with the query is a necessary and sufficient condition for them receiving the same attention weight. On the other hand, if we have two queries attending to the same key, these having the same dot product with this key is neither a sufficient nor a necessary condition for these queries to assign the same attention weight to the key, as the assigned weight depends on what

dot products the queries have with the other keys they are attending to. We have not yet derived a metric for this case.

The other case is when we assign the top- k key-clusters by some metric to each query-cluster, we want the independent *top- k key-cluster selections* of the queries within each cluster to be as similar as possible, i.e. the ranking matters more than the actual scores. From this observation, we can derive the representation of query q as

$$r(q) = \frac{R_k q}{|R_k q|}, \quad (9)$$

where R_k is the root matrix of $K_c^T K_c$, formed from the centroids of the key clusters instead of the actual keys. In short, the derivation consists of four steps. The first is to represent each query with the sign of its dot product difference, for each possible ordered pair of key-centroids. This is 1 if it has a higher dot product with the first key-centroid in the pair, and -1 otherwise. The second is relaxing this to the difference in dot products, and the third is showing that after the relaxation this represents a linear transformation D such that $D^T D \propto K_c^T K_c$. The final step is noting that the scale invariance in q that was lost in the relaxation can be recovered by normalizing the final representation. The full argumentation is found in Appendix A.

Complexity The operations are in both cases forming a metric matrix through a matrix multiplication of order $\mathcal{O}(nd^2)$ (or $\mathcal{O}((n/c)d^2)$ if using centroids), performing eigenvalue decomposition on it which has order $\mathcal{O}(d^3)$, forming the root matrix which is a small operation, and projecting the vectors through it which is $\mathcal{O}(nd^2)$. Overall, the complexity is

$$\mathcal{O}(nd^2 + d^3) \quad (10)$$

3.2.3 Softmax-aware clustering

A further extension of the target-space clustering is using nonlinear transformations to increase the rank of the representations, while at the same time aligning them even more closely with the downstream usage. This is an idea under development, that is described in Appendix B. It has not been used for any of the evaluations in this report.

3.3 Cluster assignment

3.3.1 Scoring

The basis for the cluster assignment is scoring each cluster of queries against all the clusters of keys. The simplest way to do this is to dot the centroids of each cluster against each other. Due to linearity, the resulting quantity is exactly the average dot product between the vectors in the clusters, so the average logit. For two clusters, $\mathcal{K}$ and $\mathcal{Q}$,

$$\bar{q}^\top \bar{k} = \left(\frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} q \right)^\top \left(\frac{1}{|\mathcal{K}|} \sum_{k \in \mathcal{K}} k \right) = \frac{1}{|\mathcal{Q}||\mathcal{K}|} \sum_{q \in \mathcal{Q}} \sum_{k \in \mathcal{K}} q^\top k. \quad (11)$$

However, due to the nonlinearity of softmax, ranking clusters by this quantity is not exactly representative of their relative post-softmax scores. The unnormalized (over keys) attention score between a key and a query (with $\sqrt{d}$ -normalization) is $e^{q^\top k/\sqrt{d}}$. Normalization over keys clearly does not change the ranking of scores. Using Jensen’s inequality for averages, and noting that the exponential function is strictly convex, we see

$$\frac{1}{|\mathcal{Q}||\mathcal{K}|} \sum_{q \in \mathcal{Q}} \sum_{k \in \mathcal{K}} e^{q^\top k/\sqrt{d}} \geq e^{\bar{q}^\top \bar{k}/\sqrt{d}}. \quad (12)$$

The equality only happens when all pairwise dot products are the same, for example when vectors in each of the clusters are the same. In slightly hand-waving terms, the more spread out a cluster is, the larger the expected underestimation using only centroids for scoring. To be more precise, the direction of spread also matters. There are ways to compensate for this, but the tried approaches did not give any major improvements. They were therefore excluded for simplicity.

3.3.2 Selection

After scoring, several selection strategies may be employed. The most straight-forward is selecting the top- k ranked key clusters for each query cluster. Another, more data-driven, method is to estimate the actual amount of attention mass for each key cluster, and greedily select the most massive clusters until an approximate attention mass threshold is passed. This is more adaptive than top- k , but also depends more heavily on the scoring being accurate.

3.3.3 Complexity

The time complexity of the assignment is

$$\mathcal{O} \left(d \left(\frac{n}{n_{cluster}} \right)^2 + \left(\frac{n}{n_{cluster}} \right)^2 \right), \quad (13)$$

with terms for scoring and cluster selection (a sorting or partitioning problem). The scoring constant depends on whether just centroids, diagonal variance compensation, or some other scoring method is used. In practice, the selection is found to dominate. As can be seen, this is quadratic in n . However, for token counts, the constant term is so much smaller than for dense attention, that this term does not dominate the attention latency.

4 Results

ClusterAttention has only been tested to a limited extent. Two experiments were performed, where one gave strong results, and the other weak. The strong results came from application to DINOv2, a vision-transformer. The weaker results came from TabPFN-3.

4.1 DINOv2-L

We evaluate ClusterAttention on the pictures in Figure 1. As a reference, we use SpargeAttn [1]. We also include normal dense attention, and dense attention using the fast quantized kernel from SpargeAttn.



(a) Image taken by Giles Laurent [13], here downscaled. Original resolution is 5302 by 6627 pixels.

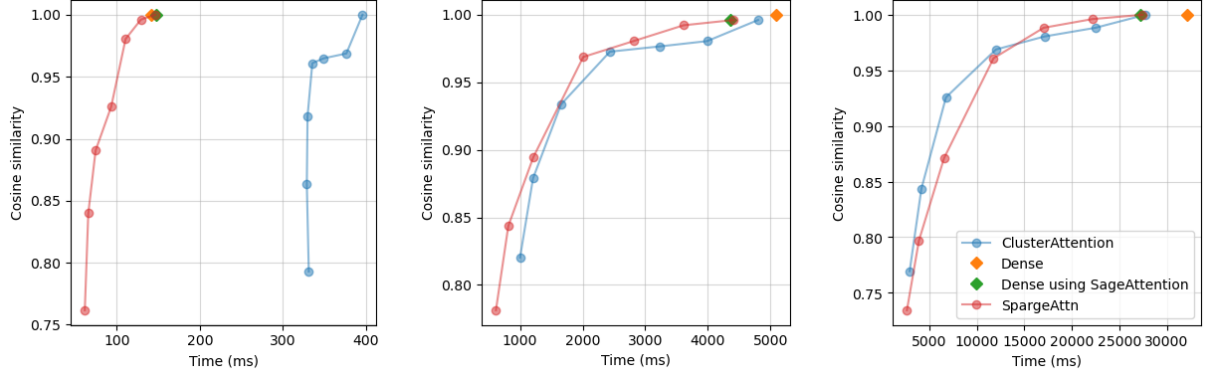


(b) Image taken by ArildV [14], here downscaled. Original resolution is 8110 by 6082 pixels.

Figure 1: Images used in evaluation. For use with DINOv2, they are square-cropped and rescaled to different resolutions.

Figures 2 and 3 show the Pareto-fronts of latency vs. representation-distortion over all the tokens, measured as average embedding cosine similarity with dense, for different image sizes. We observe that, when the images are downscaled to around 20,000 tokens, routing overhead makes ClusterAttention slow compared to SpargeAttn and the dense attentions. At the native resolution of Figure 1a with around 150,000 tokens, ClusterAttention becomes comparable with SpargeAttn for part of the front, notably providing lower distortion at sparsities over 50%. With the images upscaled to have 360,000 tokens, ClusterAttention is Pareto-better for arguably most of the front.

Figure 4 shows the latency scaling of ClusterAttention with the number of tokens, with a breakdown over its parts, as well as a fitted line in log-log-space between the last two measurements. Here, the number of tokens attended to was fixed to 64 clusters of 64 tokens, so 8192 tokens. This simulates deployment on a model with sparse attention patterns, or that in general works well with highly sparse attention. An example of such a model is Deepseek V3.2 [15], which attends to a set 2048 tokens shared over all heads in each



(a) Downscaled to 2072 by 2072 pixels, meaning 21,904 tokens. Average over 3 runs. (b) Slightly upscaled to 5306 by 5306 pixels, meaning 143,641 tokens. Average over 3 runs. (c) Upscaled to 8400 by 8400 pixels, meaning 360,000 tokens. Single-run measurements.

Figure 2: Pareto front on latency vs. representation-distortion, measured in average cosine-similarity with dense. Performed on DINOv2, with the image in Figure 1a. The points correspond to attending to 0.05, 0.1, 0.2, 0.4, 0.6, 0.8, 1.0 of all the clusters.

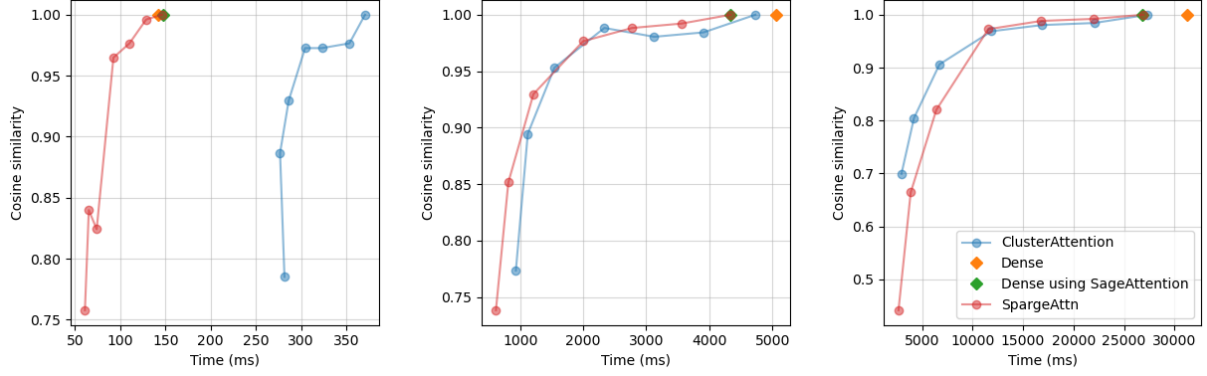
layer. We find that clustering overhead is approximately flat up to around 100,000 tokens, where it starts curving upward, and between 562,500 and 1 million tokens grows slightly below linearly. At around the same point, the actual attention computation catches up, although where this happens is a direct function of how many tokens we select in the top- k . As expected, it grows linearly with token count. Cluster assignment grows at a rate approaching quadratic, but due to its small constant it is only around 11% of total attention latency at 1 million tokens.

4.2 TabPFN-3

TabPFN-3 is the latest in the series of tabular foundation models from Prior Labs [2]. We tested applying ClusterAttention to this model, in the context of 8 large classification dataset from the TALENT benchmark [16]. ClusterAttention did not appear to find meaningful clusters with low internal variance. We also found that the attention distributions of the model were quite uniform, meaning that it is not well suited for sparse attention. ClusterAttention did not significantly outperform random subsampling as a method for sparse routing on this model, however, it is unclear whether oracle routing does either. The model was, however, rather resilient to purely random subsampling of keys. In most cases it introduced only a slight degradation, although there were also large degradations as well as cases where accuracy improved.

5 Discussion

Not enough testing has been conducted to make any conclusive statements about performance. However, broadly, the good performance on DINOv2 and failure on TabPFN-3 indicates that there is some qualitative difference in the key and query spaces produced by these models. We speculate that the random feature embeddings of TabPFN-3, and/or the often geometrically local nature of tabular data could be the difference. The image model may have more stable and structured internal representations. Further investi-



(a) Downscaled to 2072 by 2072 pixels, meaning 21,904 tokens. Average over 3 runs. (b) Downscaled to 5306 by 5306 pixels, meaning 143,641 tokens. Average over 3 runs. (c) Upscaled to 8400 by 8400 pixels, meaning 360,000 tokens. Single-run measurements.

Figure 3: Pareto front on latency vs. representation-distortion, measured in average cosine-similarity with dense. Performed on DINOv2, with the image in Figure 1b. The points correspond to attending to 0.05, 0.1, 0.2, 0.4, 0.6, 0.8, 1.0 of all the clusters.

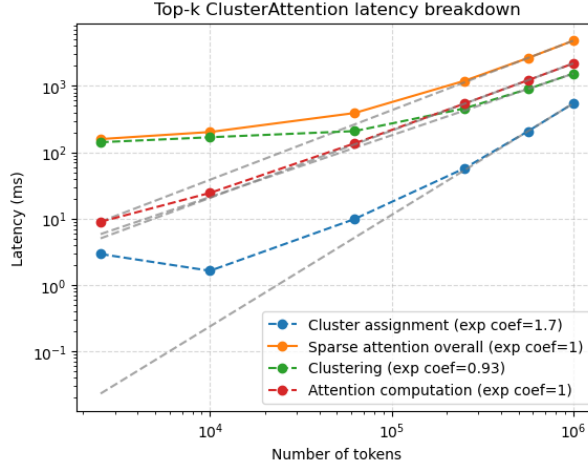


Figure 4: Latency scaling for ClusterAttention with top- k assignment, and k fixed to 64 blocks of 128 keys. Timing averages over 3 runs on the image in Figure 1b.

gation is needed to clarify what causes the failure to form meaningful clusters, and to understand if it is a fault in the clustering method or a property of the model.

ClusterAttention has noticeable routing overhead. This is expected, as the method has been developed with a focus on accurate routing for high token-counts. Nevertheless, there may be minor algorithmic modifications, optimizations to the implementation code, that could cut down on this overhead.

We note that on the tested images, ClusterAttention appears to not converge smoothly to the output of dense attention as the amount of sparsity nears zero. Although this occurs at lower levels of sparsity, and the method is generally intended for use in more sparse settings, it may be worth investigating if it occurs in other images. We speculate that it might be because of a set of clusters with high internal variance, that are not found by the routing, as SpargeAttn does not have this problem. If this is the case, it might be alleviated by the variance compensation discussed in Subsection 3.3.1, although this has

not been tested.

6 Further work

Clustered attention [6] show that the error from their approximation is bounded as a function of the Euclidean distance between the queries in each cluster. It is possible that this idea could be combined with ClusterAttention, in the sense of not necessarily excluding unselected key-clusters completely. For example, there may exist a similar bound for representing the unselected clusters by their key-centroid and average value, interacting with the query-cluster centroid. This could either be used directly in combination with the key-cluster selection of ClusterAttention, or some bound could be derived based on the variance within the clusters, such that thresholding of which key-clusters to perform per-token attention on could be performed. Intuitively, for a key-cluster and query-cluster pair, the larger the within-cluster variance for each of the queries, keys, and values is, and the higher the attention mass assigned by the queries in the query cluster to the keys in the key-cluster, the larger the expected error from representing this interaction with the centroids.

Given this observation about the approximation error, it is likely that fixed cluster sizes are not optimal. Dense regions in key or query space may be able to use larger clusters, while sparser regions benefit from smaller clusters. By allowing different sizes through adaptive stopping of the recursive splitting, the budget on the number of clusters may be better used. This could also be combined with splits that are not as close to the middle as possible, but are instead placed to maximize the reduction in spread for the newly formed clusters. This does not need to conflict with creating clusters of sizes that are powers of 2.

References

- [1] J. Zhang, C. Xiang, H. Huang, J. Wei, H. Xi, J. Zhu, and J. Chen, “SpargeAttention: Accurate and Training-free Sparse Attention Accelerating Any Model Inference,” 2025.
- [2] L. Grinsztajn, K. Flöge, O. Key, F. Birkel, P. Jund, B. Roof, M. Manium, S. B. Hoo, M. Bühler, A. Garg, D. Safaric, J. Robertson, B. Jäger, S. Alessi, A. Hayler, V. Moroshan, L. Purucker, P. Singer, A. Arazi, J. Siems, J. H. Metzen, G. Grab, N. Erickson, S. Guo, E. Kalfon, S. Bing, D. Salinas, C. Cornu, L. C. Wehrhahn, D. Kriuchkova, K. Kaya, L. Sidhoum, M. Salmon, J. Chen, M. Hulsebos, Y. LeCun, S. Müller, B. Schölkopf, S. Gambhir, N. Hollmann, and F. Hutter, “TabPFN-3: Technical Report,” 2026.
- [3] H. Xu, N. Usuyama, J. Bagga, S. Zhang, R. Rao, T. Naumann, C. Wong, Z. Gero, J. González, Y. Gu, Y. Xu, M. Wei, W. Wang, S. Ma, F. Wei, J. Yang, C. Li, J. Gao, J. Rosemon, T. Bower, S. Lee, R. Weerasinghe, B. J. Wright, A. Robicsek, B. Piening, C. Bifulco, S. Wang, and H. Poon, “A whole-slide foundation model for digital pathology from real-world data,” *Nature*, vol. 630, no. 8015, pp. 181–188, 2024.
- [4] T. Hu, J. Zhang, Z. Su, and R. Yi, “UltraGen: High-Resolution Video Generation with Hierarchical Attention,” 2025.
- [5] E. Nguyen, M. Poli, M. Faizi, A. Thomas, C. Birch-Sykes, M. Wornow, A. Patel, C. Rabideau, S. Massaroli, Y. Bengio, S. Ermon, S. A. Baccus, and C. Ré, “HyenaDNA: Long-Range Genomic Sequence Modeling at Single Nucleotide Resolution,” 2023.
- [6] A. Vyas, A. Katharopoulos, and F. Fleuret, “Fast Transformers with Clustered Attention,” 2020.
- [7] H. Tan, S. Wang, Y. Qiao, J. Zhang, Y. Bai, P. Gong, Z. Jin, and C. Li, “AdaCluster: Adaptive Query-Key Clustering for Sparse Attention in Video Generation,” 2026.
- [8] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, “Quest: Query-aware sparsity for efficient long-context llm inference,” 2024.
- [9] J. Luo, J. Chen, J. Wang, C. Wang, H. Zhu, Q. Sun, C. Gao, Z. Chen, and J. Li, “Attention Sparsity is Input-Stable: Training-Free Sparse Attention for Video Generation via Offline Sparsity Profiling and Online QK Co-Clustering,” 2026.
- [10] N. Hollmann, S. Müller, K. Eggenberger, and F. Hutter, “TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second,” 2023.
- [11] J. Qu, D. Holzmüller, G. Varoquaux, and M. L. Morvan, “TabICL: A Tabular Foundation Model for In-Context Learning on Large Data,” 2025.
- [12] D. Boley, “Principal Direction Divisive Partitioning,” *Data Mining and Knowledge Discovery*, vol. 2, no. 4, pp. 325–344, 1998.
- [13] G. Laurent, “Marsh deer and Pink Ipê tree in Encontro das Águas State Park,” 2024. Licensed under CC BY-SA 4.0, via Wikimedia Commons.

- [14] ArildV, “Hamngatan October 2025,” 2025. Licensed under CC BY-SA 4.0, via Wikimedia Commons.
- [15] DeepSeek-AI, A. Liu, A. Mei, B. Lin, B. Xue, B. Wang, B. Xu, B. Wu, B. Zhang, C. Lin, C. Dong, C. Lu, C. Zhao, C. Deng, C. Xu, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, E. Li, F. Zhou, F. Lin, F. Dai, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Li, H. Liang, H. Wei, H. Zhang, H. Luo, H. Ji, H. Ding, H. Tang, H. Cao, H. Gao, H. Qu, H. Zeng, J. Huang, J. Li, J. Xu, J. Hu, J. Chen, J. Xiang, J. Yuan, J. Cheng, J. Zhu, J. Ran, J. Jiang, J. Qiu, J. Li, J. Song, K. Dong, K. Gao, K. Guan, K. Huang, K. Zhou, K. Huang, K. Yu, L. Wang, L. Zhang, L. Wang, L. Zhao, L. Yin, L. Guo, L. Luo, L. Ma, L. Wang, L. Zhang, M. S. Di, M. Y. Xu, M. Zhang, M. Zhang, M. Tang, M. Zhou, P. Huang, P. Cong, P. Wang, Q. Wang, Q. Zhu, Q. Li, Q. Chen, Q. Du, R. Xu, R. Ge, R. Zhang, R. Pan, R. Wang, R. Yin, R. Xu, R. Shen, R. Zhang, S. H. Liu, S. Lu, S. Zhou, S. Chen, S. Cai, S. Chen, S. Hu, S. Liu, S. Hu, S. Ma, S. Wang, S. Yu, S. Zhou, S. Pan, S. Zhou, T. Ni, T. Yun, T. Pei, T. Ye, T. Yue, W. Zeng, W. Liu, W. Liang, W. Pang, W. Luo, W. Gao, W. Zhang, X. Gao, X. Wang, X. Bi, X. Liu, X. Wang, X. Chen, X. Zhang, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yu, X. Li, X. Yang, X. Li, X. Chen, X. Su, X. Pan, X. Lin, X. Fu, Y. Q. Wang, Y. Zhang, Y. Xu, Y. Ma, Y. Li, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Qian, Y. Yu, Y. Zhang, Y. Ding, Y. Shi, Y. Xiong, Y. He, Y. Zhou, Y. Zhong, Y. Piao, Y. Wang, Y. Chen, Y. Tan, Y. Wei, Y. Ma, Y. Liu, Y. Yang, Y. Guo, Y. Wu, Y. Wu, Y. Cheng, Y. Ou, Y. Xu, Y. Wang, Y. Gong, Y. Wu, Y. Zou, Y. Li, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Z. F. Wu, Z. Z. Ren, Z. Zhao, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Gou, Z. Ma, Z. Yan, Z. Shao, Z. Huang, Z. Wu, Z. Li, Z. Zhang, Z. Xu, Z. Wang, Z. Gu, Z. Zhu, Z. Li, Z. Zhang, Z. Xie, Z. Gao, Z. Pan, Z. Yao, B. Feng, H. Li, J. L. Cai, J. Ni, L. Xu, M. Li, N. Tian, R. J. Chen, R. L. Jin, S. S. Li, S. Zhou, T. Sun, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Song, X. Zhou, Y. X. Zhu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Z. Huang, Z. Xu, Z. Zhang, D. Ji, J. Liang, J. Guo, J. Chen, L. Xia, M. Wang, M. Li, P. Zhang, R. Chen, S. Sun, S. Wu, S. Ye, T. Wang, W. L. Xiao, W. An, X. Wang, X. Sun, X. Wang, Y. Tang, Y. Zha, Z. Zhang, Z. Ju, Z. Zhang, and Z. Qu, “DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models,” 2025.
- [16] S.-Y. Liu, H.-R. Cai, Q.-L. Zhou, and H.-J. Ye, “TALENT: A Tabular Analytics and Learning Toolbox,” 2024.

A Query-representation for key-cluster ranking

A good representation of a query is the relative ranking of the key-clusters by the dot-product with their centroids. For simplicity of notation, we do the derivation using keys (k) and not key-centroids ($c(\mathcal{K})$), but note that using centroids is more aligned with the goal of clustering, and also empirically gives slightly better performance. Formally, with this adjustment for clarity, the representation is

$$r_{(a,b)}(q) = \text{sign}((K_a - K_b) \cdot q), \quad (14)$$

where single indexing into the key-matrix gives a single key, and the representation has one value for each pair from the space of ordered pairs of key-indices with size n^2 . From here, we relax to the actual distances, so

$$r_{(a,b)}(q) = (K_a - K_b) \cdot q \quad (15)$$

noting that we lose scale invariance of q . However, we still have offset invariance over the keys, which is desirable for comparing rankings. We can note that this representation lives in $\mathbb{R}^{n^2}$, but it is found through a linear transformation, so it can have at most rank d . Precisely, the linear transformation is

$$D_{(a,b),m} = K_{a,m} - K_{b,m}. \quad (16)$$

where we impose some ordering over the pairs. Since all geometry in transformed space involves forming $D^T D$, we can study this matrix to find a non-redundant representation. We find

$$\begin{aligned} (D^T D)_{jk} &= \sum_l D_{lj} D_{lk} = \sum_{(a,b)} (K_{a,j} - K_{b,j})(K_{a,k} - K_{b,k}) = \\ &\quad \sum_{(a,b)} (K_{a,j} K_{a,k} - K_{a,j} K_{b,k} - K_{b,j} K_{a,k} + K_{b,j} K_{b,k}) = \\ &= n \sum_a K_{a,j} K_{a,k} - \left(\sum_a K_{a,j} \right) \left(\sum_b K_{b,k} \right) - \left(\sum_b K_{b,j} \right) \left(\sum_a K_{a,k} \right) + n \sum_b K_{b,j} K_{b,k}. \end{aligned} \quad (17)$$

We then note that

$$\sum_a K_{a,j} K_{a,k} = (K^T K)_{jk} \quad (18)$$

and similar for the sum over $K_{b,j} K_{b,k}$, while

$$\left(\sum_a K_{a,j} \right) \left(\sum_b K_{b,k} \right) = (n \bar{K}_{\cdot,j})(n \bar{K}_{\cdot,k}) = n^2 \bar{K}_{\cdot,j} \bar{K}_{\cdot,k}, \quad (19)$$

and similar for the other factored sum, where indexing with $\cdot, j$ indicates a *column* of the key matrix. Putting it together, we find

$$(D^T D)_{jk} = 2n(K^T K)_{jk} - 2n^2 \bar{K}_{\cdot,j} \bar{K}_{\cdot,k} = 2n(K_c^T K_c)_{jk}, \quad (20)$$

which is the $2n$ times j, k th element of the covariance matrix over the keys. So, using this (or actually the covariance matrix over the key-cluster centroids), gives the same

geometry up to a scalar factor. Finally, to recover the scaling invariance over q we had in the original ranking metric, we note that scaling q directly scales the representation. So we can simply normalize the representation. With R_k as the root matrix of $K_c^T K_c$, we get our key-aware query representations as

$$r(q) = \frac{R_k q}{|R_k q|}. \quad (21)$$

B Softmax-aware clustering

This appendix describes an idea that is under development. We can rephrase the query-space clustering argument as follows. We consider a representation of k_1 that is query-aware as

$$r(k_1) = \frac{Q}{\sqrt{n}} k_1, \quad (22)$$

i.e., k_1 is represented as its dot with all the queries (the logits). This representation lies in $\mathbb{R}^n$, but being a projection from $\mathbb{R}^d$ it occupies a subspace of at most dimensionality d . In fact, as attention key often only occupy a subspace of lower dimensionality than their intrinsic dimensionality, the representation likely occupies a subspace of lower dimensionality than d . We can see that we recover the same (pseudo-)metric M as before in this space, since

$$\begin{aligned} \|r(k_1) - r(k_2)\|^2 &= (r(k_1) - r(k_2))^T (r(k_1) - r(k_2)) = \\ &= \left(\frac{Q}{\sqrt{n}} k_1 - \frac{Q}{\sqrt{n}} k_2 \right)^T \left(\frac{Q}{\sqrt{n}} k_1 - \frac{Q}{\sqrt{n}} k_2 \right) = (k_1 - k_2)^T \frac{Q^T Q}{n} (k_1 - k_2). \end{aligned} \quad (23)$$

To avoid the representation becoming low-rank, we can note that the logits will be passed through a softmax function as part of the attention computation. To mirror this, we apply the softmax transform over the keys for each query. This is similar to the kernel-trick in support-vector machines, but with the additional benefit of directly mirroring the downstream usage of the clusters. The specific nonlinearity of softmax means that for a given query, the difference between two larger logits carries more meaning, than the same difference between two smaller logits. Thus, clustering in a *softmax aware* space, by applying a softmax operation over the keys (whether clustering keys or queries), emphasizes tight clustering of keys in regions of the space where they significantly affect the attention values. It can also increase the number of meaningful directions/coordinates to cluster along.

Of course, performing this projection is essentially calculating full attention, which is what we want to avoid. To avoid this cost, we can choose a subset of queries as landmarks. This can be done, for example, by random subsampling. We can also project this new representation onto its principal components. There is a risk that the strong selectivity of softmax in this subsampled space could collapse the representations of keys. To mitigate this, the softmax temperature may be increased.

C Computational complexities

C.1 Basic recursive splitting

The number of operations in the dominant parts of split i is proportional to

$$2^i \left(\underbrace{pd^2}_{\text{power iteration}} + \underbrace{\frac{n}{2^i}d}_{\text{projection}} + \underbrace{\frac{n}{2^i}}_{\text{partitioning}} \right) = 2^i pd^2 + nd + n. \quad (24)$$

This is repeated for i from 0 to $\log_2(n/c) - 1$, roughly, since n/c is not always a power of 2. We get for the power iteration

$$\sum_{i=0}^{\log_2(n/c)-1} 2^i pd^2 = pd^2 \sum_{i=0}^{\log_2(n/c)-1} 2^i = pd^2 \left(\frac{n}{c} - 1 \right) \quad (25)$$

and the algorithmic complexity as

$$\mathcal{O} \left(pd^2 \frac{n}{c} + nd \log_2(n/c) \right). \quad (26)$$

As can be seen, the splitting is dominated by the $n \log_2 n$ term in the limit of n . In practice, all terms can be meaningful, due to the roughly linear scaling but different constants.

C.2 Diagonalized recursive splitting

The number of operations is proportional to

$$\underbrace{nd^2 + d^3}_{\text{PC decomposition}} + 2^i \left(\underbrace{\frac{d}{2^i}d}_{\text{variance estimation}} + \underbrace{\frac{n}{2^i}}_{\text{partitioning}} \right). \quad (27)$$

Here, subsampling to d vectors was assumed. The PC decomposition consists of forming second moment matrices from a set of vectors, projecting a set of vectors (both $\mathcal{O}(nd^2)$), and eigendecomposing matrices ($\mathcal{O}(d^3)$). This gives a complexity of

$$\mathcal{O}(nd^2 + d^3), \quad (28)$$

although the partitioning at $\mathcal{O}(n)$ tends to be the largest cost for the commonly seen d .